AGENT ARCHITECTURE · WITH CLAUDE · 2026

How to build an *effective* agent harness for code review.

...and which SDK to reach for when you do.

SUBJECT
What goes inside the system that wraps the model.

EXAMPLE
Explained through a code-*review* agent.

agent = *model* + *harness.*

What this *session* will be about.

SIX STOPS
MOTIVATION → METHODOLOGY → SHIP



· START · FIELD GUIDE OPENS · SIX BEATS · ONE HARNESS · SCAN & SHIP ·

MOTIVATION

“All you need is a *good prompt*.”

— *some people say*
github.com/obra/superpowers

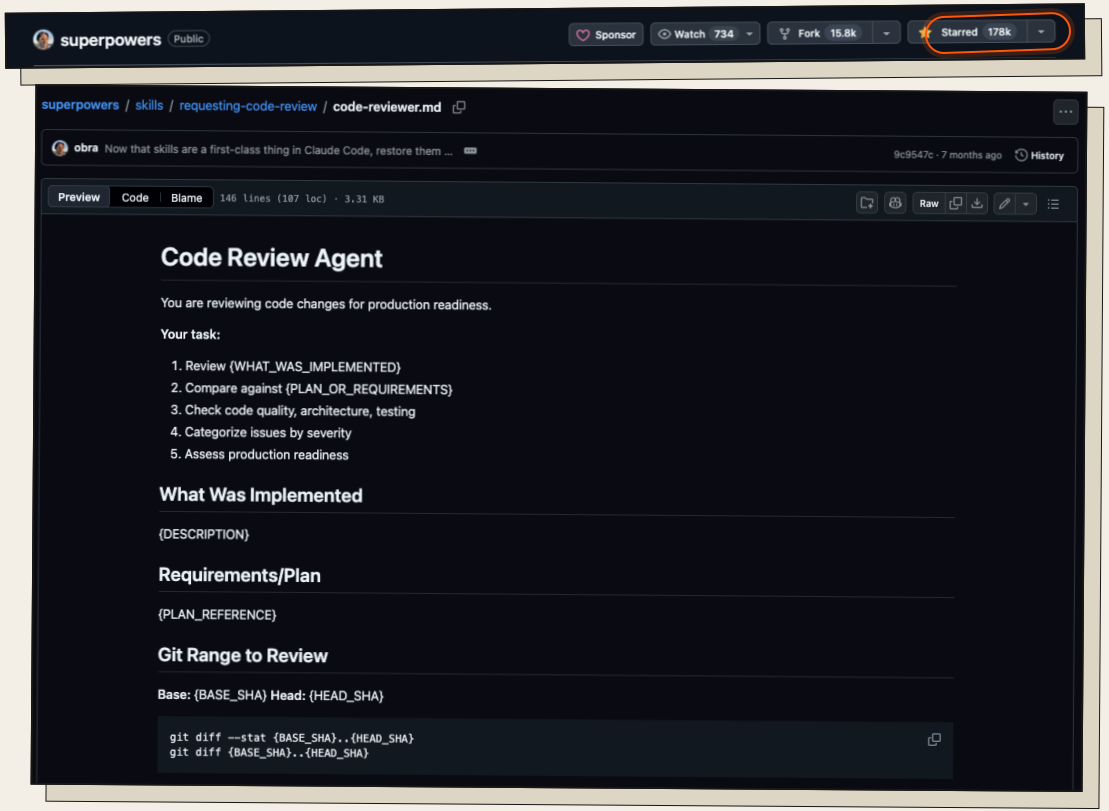


EXHIBIT A · OBRA / SUPERPOWERS

CODE-REVIEWER.MD · 146 LINES

WHY CONTEXT MATTERS

Outside-diff *impact slicing.*

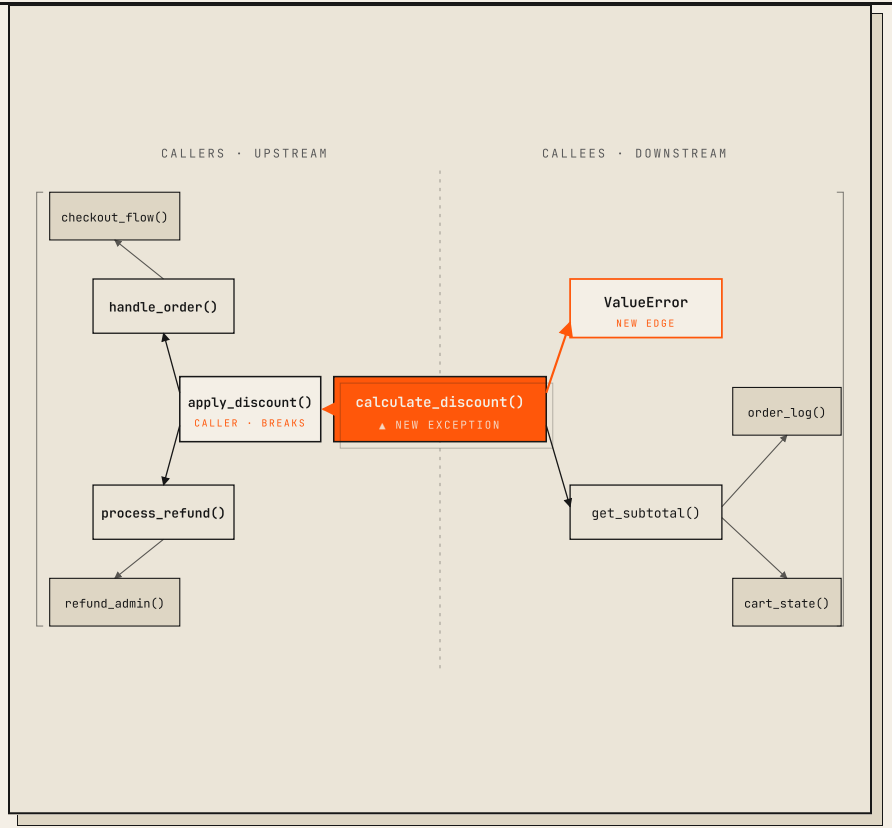
Many bugs hide **outside the diff**. A useful review needs the *call-graph slice* — every caller and callee touched by the change — not just the lines that turned red and green.

```python
def calculate_discount(price, discount_percent, min_purchase):
    if discount_percent < 0 or discount_percent > 100:
        raise ValueError("Discount must be between 0 and 100")
    if price < min_purchase:
        return price  # No discount if below minimum
    return price * (1 - discount_percent / 100)


def apply_discount(self, discount_percent):
    """Apply a discount to the order subtotal."""
    subtotal = self.get_subtotal()
    return calculate_discount(subtotal, discount_percent)
```

EXHIBIT · DIFF UNDER REVIEW

+ RAISE VALUEERROR — NEW EXCEPTION PATH



▲ New exception added
A NEW RAISE INSIDE THE DIFF — THE ORANGE NODE

▼ Caller breaks silently
CALLERS UP THE GRAPH NEVER EXPECTED TO HANDLE IT

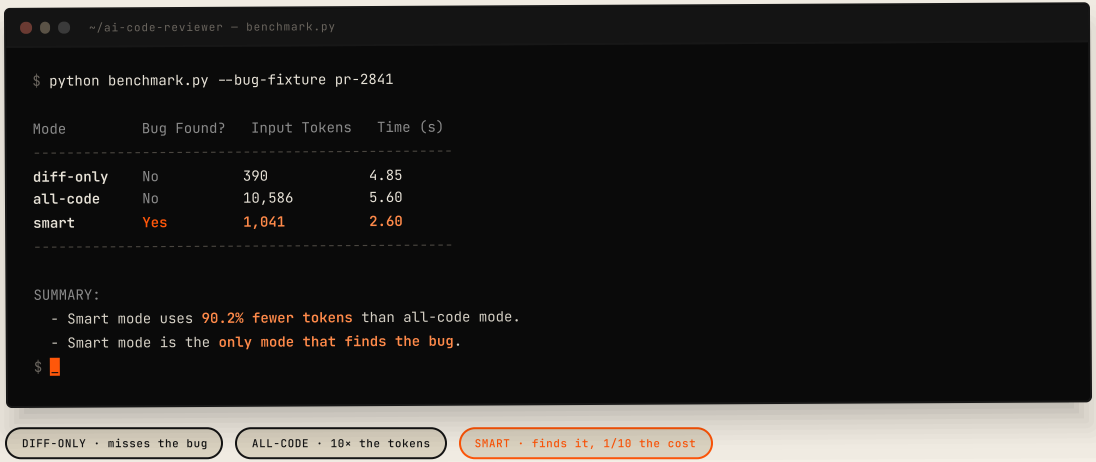
RECEIPTS

Let me *prove* it to you.

Same task, same model, same prompt. The only thing that changes is *the harness around it.*

A "smart" harness picks the right context, runs the right tool, and gets the right answer — for a fraction of the tokens.

SOURCE · github.com/Hendrik040/ai-code-reviewer



HOW WE GOT HERE

What does a senior SWE need to do a *good* job?

Work backwards *from the human.*

WHAT THE SENIOR SWE NEEDS	AGENT DESIGN PRIMITIVE
Access to the *codebase*	**Computer + Sandbox** — FILESYSTEM · ISOLATED ENV
Read & write code, *run scripts*	**Bash + Code Execution** — GENERAL-PURPOSE ACTION SURFACE
Tools for the job — *tests, linters*	**Curated tools + MCP** — SMALL, SHARP, TARGETED
Context *codebase & the job to be done*	**Skills, learnings & memory** — CONTEXT ENGINEERING

Each harness primitive derives from something the model can't deliver on its own.

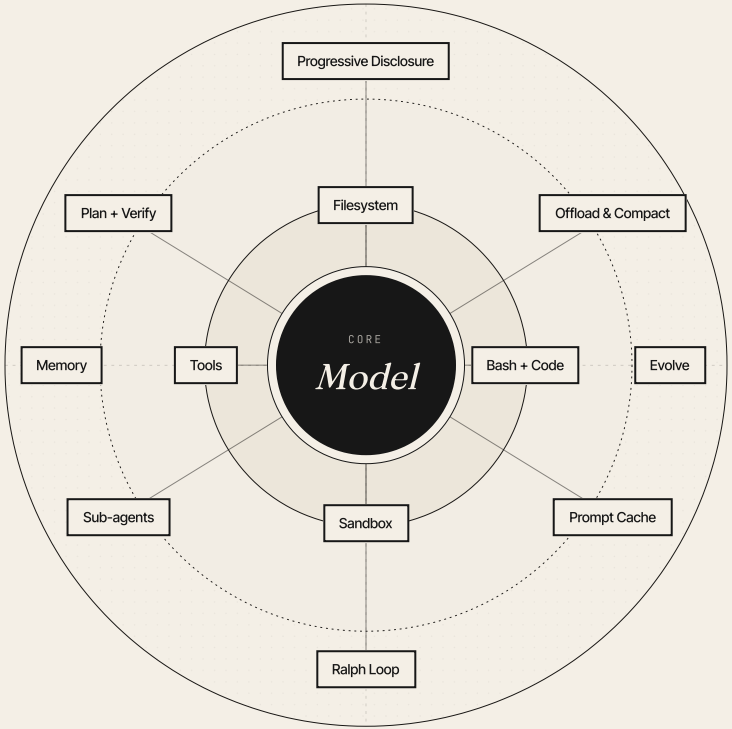
THE FULL PICTURE

The harness, *at a glance.*

The **core** is the model.

The **inner ring** is the computer we hand it: filesystem, shell, sandbox, a small set of curated tools.

The **outer ring** is everything we layer on top to keep work coherent: context engineering, orchestration, learning.

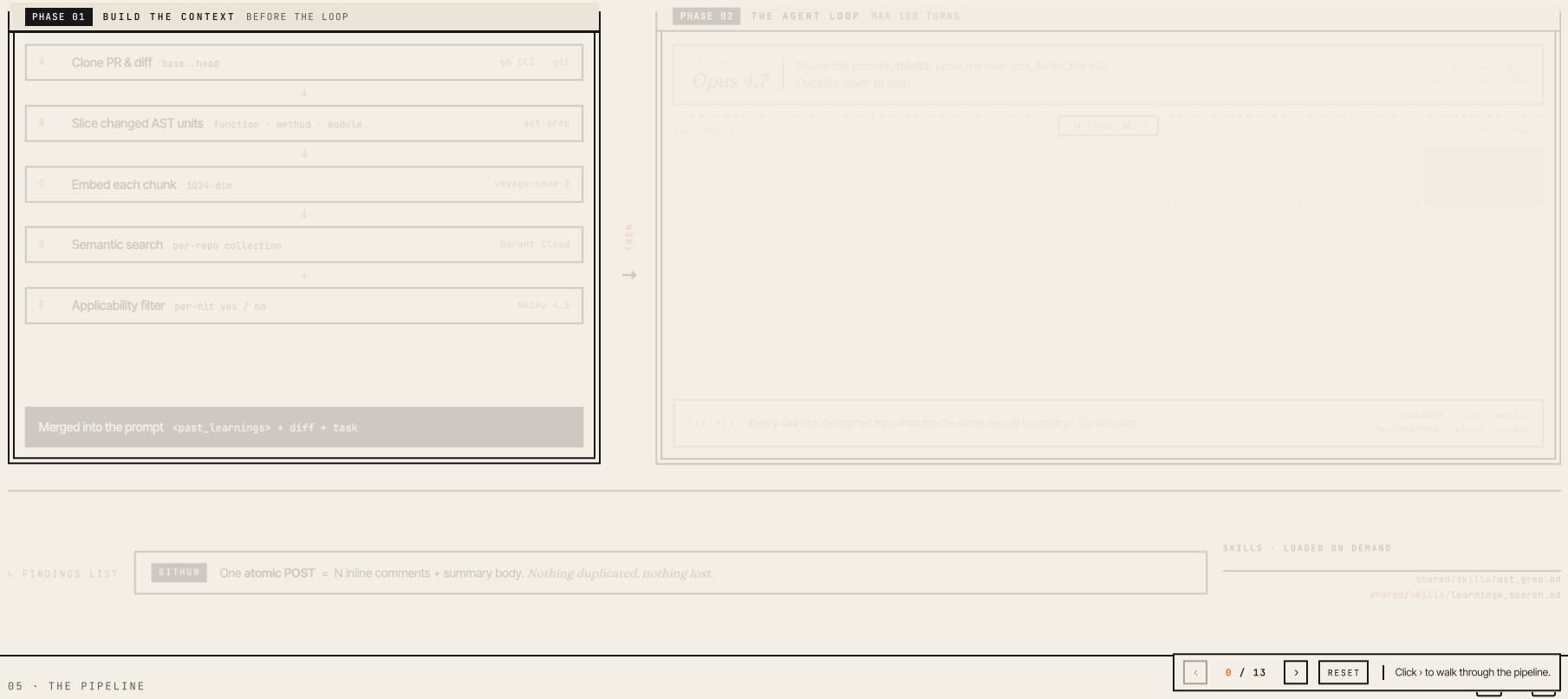


CLICK ANY NODE · SEE HOW WE BUILT IT

HOW IT ACTUALLY RUNS

The reviewer, *end-to-end.*

Two phases. Build a context-rich prompt up front, then hand it to a model that drives a small toolkit until it submits findings.



TWO HARNESSES, ONE MODEL

Two SDKs, *same* model.

*Both call **Opus 4.7** through the same Anthropic API. The only thing that changes is who owns the loop.*

A CLIENT SDK — EXPLICIT

~30
lines we drive every turn

We drive every turn. **Full control** over flow, caching, dispatch.

WE MANAGE

- The **while** loop
- Parsing **tool_use** blocks
- Dispatching tool fns
- Building **tool_result**
- Prompt-cache breakpoints
- Stop conditions

SDK MANAGES

- HTTP / auth
- Streaming framing

CODE SHAPE — PYTHON

```python
# we own the loop
messages = [{"role":"user", "content":prompt}]
while num_turns < MAX_TURNS:
    response = client.messages.create(
        model=MODEL, tools=ALL_TOOLS,
        system=SYSTEM, messages=messages,
        output_config={"effort": effort},
    )
    if response.stop_reason ≠ "tool_use":
        break
    for block in response.content:
        if block.type == "tool_use":
            result = _dispatch(block.name, block.input)
            messages.append(_tool_result(result))
```

VS

B AGENT SDK — DECLARATIVE

~3
lines — the SDK runs the loop

We declare tools. **The SDK runs the loop** & manages caching.

WE DECLARE

- **@tool** decorators
- **allowed_tools** list
- effort, max_turns

SDK MANAGES

- The **while** loop
- Tool dispatch (MCP)
- **Cache** breakpoints
- Stop conditions

CODE SHAPE — PYTHON

```python
@tool("read_file_section", "Read N lines", {...})
async def read_file_section(args): ...
# (six more @tool decorators)

server = create_sdk_mcp_server(
    name="reviewer",
    tools=[read_file_section, ...],
)
options = ClaudeAgentOptions(
    model=MODEL, system_prompt=SYSTEM,
    mcp_servers={"reviewer": server},
    allowed_tools=["mcp__reviewer__*", ...],
    max_turns=MAX_TURNS, effort=effort,
)
async for msg in query(prompt, options):
    log(msg)  # SDK runs the loop
```

“ *Same model, same tools — the only thing that changes is who owns the loop.*

2×
CHEAPER · SAME RECALL

OPEN SOURCE & REPRODUCIBLE

Build your own, *this afternoon.*

Everything in this deck is **open and reproducible**. Scan the QR or hit the link to see the full benchmark, the eval harness, and the code-review agent itself on GitHub.

We packaged the *methodology* as a **Claude Code plugin** — install the two commands on the right and Claude will walk you through building, evaluating, and iterating on a benchmark of your own.

And if you want to talk through any of it — the *why* behind every choice in this harness, or how to build one for your team — reach out on LinkedIn or X.

SCAN FOR THE GITHUB REPO · OR CONNECT WITH ME

Repo, *then* let's talk.

/in/climateadvocateaienthusiast — LINKEDIN

@HKrackDev — X / TWITTER

INSTALL THE PLUGIN — ~/ · ZSH

```
$ claude plugin marketplace add ant-open-skills/custom-model-bench
$ claude plugin install custom-model-bench@ant-open-skills
```

/01 COMPUTER LAYER

Filesystem.

A workspace that outlives the context window.

Read data and code. Offload long tool outputs. Persist a plan across restarts. **Add git** and the agent gets versioning, rollback, and a shared ledger with humans and other agents.



/02 COMPUTER LAYER

Bash + *code*.

One general-purpose tool to author all the others.

Instead of pre-configuring twenty narrow tools, give the agent a shell. It can chain CLIs, write throwaway scripts, and design new tools on the fly. **Code execution is the default strategy** for autonomous problem-solving.



/03 COMPUTER LAYER

Sandbox.

A safe place for the agent to act, observe, and iterate.

Isolated, allow-listed, network-controlled. Spawn on demand, tear down when done. **Pre-load good defaults** — language runtimes, browsers, test runners — so the agent can verify its own work without setup tax.



/04 COMPUTER LAYER

Curated tools.

A dozen, not a hundred.

Every tool definition costs tokens. Overlapping tools confuse the model. Claude Code ships with around a dozen tools; Manus uses fewer than twenty. **Push complex actions down to the shell** rather than up into the tool layer.

TOOL_REGISTRY — 12 ATOMIC

read read a file	**write** write or overwrite
edit surgical patch	**grep** search contents
ls list directory	**bash** run shell
web_search fresh knowledge	**web_fetch** read URL
spawn sub-agent	**todo** plan tracking
skill load procedure	**done** finish task

RING 2 · CONTEXT

Spend the window *wisely.*

Context is finite. Three patterns keep it from rotting.

/ 05
Progressive disclosure

Surface metadata only. Let the agent pull full definitions on demand via skills, indexes, or `--help`.

```
skill: pdf-tools
name, description loaded
body: read on demand
```

/ 06
Offload & compact

Push old tool output to disk. Summarize only when offloading hits diminishing returns — never before.



/ 07
Prompt caching

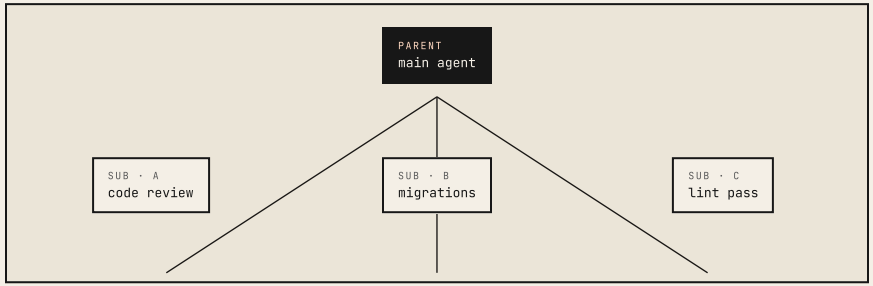
Stable prefixes mean cache hits. Cache hit rate is the production cost metric — without it, long runs are unaffordable.

```
HIT system prompt · $0.0001/Mtok
HIT tool defs · cached
MISS latest msg · full price
```

/08 ORCHESTRATION

Sub-agents.

Isolated context, parallel work, clean handoffs.



/09 ORCHESTRATION

Ralph Loop.

When one window won't fit, run many — in sequence.

1 Initializer writes plan.md →

2 Agent picks next item, works, commits ↻

3 Stop hook verifies, then reinjects goal →

4 Fresh context, plan still on disk, repeat ✓

/10 ORCHESTRATION

Plan + *verify*.

Decompose, write it down, run the tests after every step.

Planning gives a long task structure. Re-reading the plan periodically reinforces the goal. **Verification grounds the work** — pre-defined test runs as a hook, errors loop back to the model with a clean signal. No verification, no progress.



/11 LEARNING

Memory.

A file the agent reads on every start, edits as it learns.

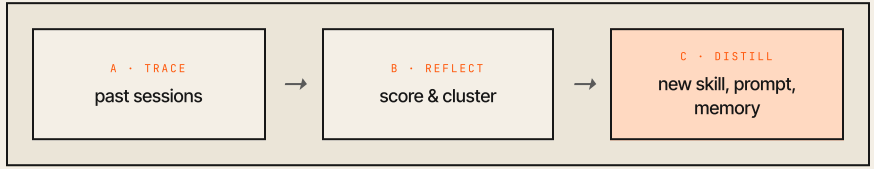
```
# AGENTS.md
## Project conventions
- Use httpx, not requests
- Test runner is pytest -q
- Lint with ruff before commit

# lessons
- Migrations need feature flags
- DB schema in /migrations
```

/12 LEARNING

Evolve.

Reflect over old runs. Distill what worked into something reusable.



PULLING IT TOGETHER

An effective harness has all *twelve*.

/01 **Filesystem** Durable workspace, offload target, shared ledger. COMPUTER	/02 **Bash + Code** One general-purpose tool the agent uses to author the rest. COMPUTER	/03 **Sandbox** Isolated runtime with browsers, runtimes, test runners. COMPUTER	/04 **Curated tools** ≈ a dozen atomic tools — not a hundred. COMPUTER
/05 **Progressive disclosure** Surface metadata; load detail only on demand. CONTEXT	/06 **Offload & compact** Tool output to disk; summarize at diminishing returns. CONTEXT	/07 **Prompt cache** Stable prefixes. Cache hit rate is the cost metric. CONTEXT	/08 **Sub-agents** Isolated windows for parallel review or migration. ORCHESTRATION
/09 **Ralph loop + hooks** Reinject goal in clean window. State on disk. ORCHESTRATION	/10 **Plan & verify** Decompose, re-read, run tests as stop-hooks. ORCHESTRATION	/11 **Memory** AGENTS.md style files. Auto-loaded, agent-edited. LEARNING	/12 **Evolve** Distill traces into new skills, prompts, memories. LEARNING